CONFLUENCE SPACE / DOCSY

Reliable Documentation Delivery with atlcli

VERSION	v8
EXPORTED	July 25, 2026
EXPORTED BY	atlcli

Contents

Reliable Documentation Delivery with atlcli

REFERENCE ARCHITECTURE

ACTIVELY MAINTAINED

DOCSY

Purpose of this reference

This page demonstrates how structured Confluence content is preserved when it is published as a production-ready PDF or Word document with atlcli. It is designed as a reproducible fidelity reference rather than a collection of unrelated visual examples.

Executive summary

Engineering documentation is most valuable when it remains readable, reviewable, and portable outside its authoring system. atlcli converts the semantic structure of a Confluence page—not a screenshot of its browser representation—into a shared document model used by its native PDF and Word renderers.

✓

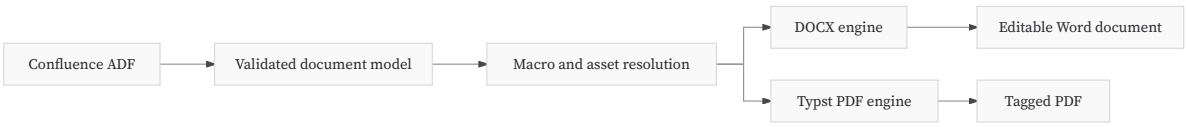
What to inspect in the exported PDF

Follow the table of contents, zoom into the vector architecture diagram, inspect the highlighted TypeScript example, and compare the status-rich control matrix across its paginated layout.

A locally executed publishing pipeline preserves structured source content as native PDF and Word documents.

Architecture at a glance

The same validated content and resolved assets feed two format-native renderers. The output formats share semantic fidelity without being reduced to a generic print view.



⚠

One content model, two native renderers

Confluence Cloud uses validated ADF as its primary source. PDF and DOCX share the decoded content and macro-resolution pipeline, then use the native capabilities of their target formats.

Fidelity principles

- Preserve authored structure before applying presentation.
- Resolve supported macros into real document blocks.
- Keep unsupported content visible and explain every degradation.
- Validate the final artifact before making it available for delivery.

[-] Why compilation stays local

The PDF compiler, WebAssembly module, templates, and fonts are bundled with the host. Page content is not sent to an external document-rendering service.

A publishing pipeline that can recover

The export runtime persists its intent before the first remote page is read. Progress is represented by a durable, versioned event protocol rather than a transient spinner.

```
const job = await exportJobs.submit({
  source: { space: "DOCSY", pageId: referencePageId },
  format: "pdf",
  scope: "page",
});

await exportJobs.watch(job.id, {
  onProgress(event) {
    console.log(event.stage, event.progress);
  },
});

```

ℹ

Shiki-powered code presentation

Code is tokenized locally with Shiki and rendered with the bundled `github-light` theme and language-aware colors. Titles, line numbers, indentation, and the complete source remain part of the exported document.

Chrome is a local execution host

Closing the side panel does not stop a browser export. Closing Chrome pauses local execution; durable work resumes when Chrome starts again.

Recovery contract

- ✓ Persist the request before source discovery
- ✓ Record stages, progress, retries, issues, and recovery events
- ✓ Fence every active runner with a lease epoch
- ✓ Resume authentication-blocked browser jobs after sign-in
- ✓ Preserve immutable history when retrying or running again

`RECOVERABLE`

`OBSERVABLE`

`LOCALLY COMPILED`

Export quality and recovery control matrix

The following deliberately wide table represents a realistic publishing operation. It combines long labels, URLs, status badges, measurements, and multi-line recovery instructions so the export must make real layout decisions.

Area	Quality gate	Input and scope	Pipeline stage	Status	Risk	Last run	Result	Recovery behavior	Evidence
Content	ADF schema validation	Confluence Cloud ADF across the complete page tree	Discover	APPROVED	LOW	2026-07-25 09:42 UTC	18 pages · 1.2 s · 0 warnings	Reject invalid input before rendering	`adf-validation-report.json`
Content	Page-tree discovery	DOCSY hierarchy through the Confluence REST API	Fetch	COMPLETE	LOW	2026-07-25 09:43 UTC	18 pages · 2.6 s · 0 warnings	Resume from the last committed page slot	Tree export contract
Content	Dynamic macro resolution	Jira, includes, datasources, and remote macro bodies	Resolve	IN PROGRESS	MEDIUM	2026-07-25 09:44 UTC	18 pages · 4.8 s · 2 warnings	Use bounded backoff, then retain a visible fallback	`macro-rendered-via`
Assets	Attachment collection	Authenticated images and	Assets	COMPLETE	LOW	2026-07-25 09:45 UTC	18 pages · 3.1 s · 0 warnings	Reuse SHA-256-addressed	`asset-summary.json`

Area	Quality gate	Input and scope	Pipeline stage	Status	Risk	Last run	Result	Recovery behavior	Evidence
		downloadable files						bytes from the safe checkpoint	
Assets	Mermaid rendering	Fenced architecture and process diagrams	Assets	APPROVED	LOW	2026-07-25 09:45 UTC	3 diagrams · 820 ms · 0 warnings	Preserve readable source for unsupported diagram types	Vector SVG plus PNG fallback
Assets	Draw.io preview resolution	Embedded Confluence preview attachments	Assets	MONITORED	MEDIUM	2026-07-25 09:45 UTC	2 diagrams · 1.4 s · 1 warning	Keep the macro body visible when a preview is missing	`macro-degraded`
Layout	Adaptive table layout	Dense 10-column operational control matrix	Render	DENSE	LOW	2026-07-25 09:46 UTC	14 rows · 310 ms · 0 warnings	Reduce padding before type size; never clip silently	`table-layout-dense`
DOCX	Native document assembly	Template, comments, captions, and shared content model	Render	COMPLETE	LOW	2026-07-25 09:46 UTC	21 pages · 2.9 s · 0 warnings	Reclaim the render reservation from prepared input	`docx-typescript`
PDF	Typst compilation	Tagged PDF from the shared document model	Render	COMPLETE	LOW	2026-07-25 09:47 UTC	21 pages · 5.7 s · 0 warnings	Restart the isolated compiler worker from prepared input	`pdf-typst`
PDF	Structure validation	Tags, language, outline, links, and embedded fonts	Validate	APPROVED	LOW	2026-07-25 09:47 UTC	21 pages · 670 ms · 0 warnings	Block emission when required structure is absent	PDF accessibility
Delivery	Atomic artifact commit	Final PDF, report, and staged artifact	Commit	COMMITTED	LOW	2026-07-25 09:48 UTC	21 pages · 140 ms · 0 warnings	Reconcile prepared finalization after a restart	SHA-256 `b0d6…8c41`
Delivery	Browser download	IndexedDB artifact for the current Atlassian site	Commit	READY	LOW	2026-07-25 09:48 UTC	21 pages · 90 ms · 0 warnings	Resume after sign-in without discarding the job	Export jobs runbook

Area	Quality gate	Input and scope	Pipeline stage	Status	Risk	Last run	Result	Recovery behavior	Evidence
Operations	Rate-limit backoff	Transient HTTP 429 responses and Retry-After policy	Waiting	WAITING	MEDIUM	2026-07-25 09:49 UTC	18 pages · 30 s · 1 retry	Wait for the bounded retry time-stamp; never du-plicate work	`rate-limited`
Operations	Retention cleanup	Deliv-ered arti-facts, re-ports, and durable history	Commit	SCHEDULE D	LOW	2026-07-25 10:00 UTC	21 pages · 75 ms · 0 warnings	Protect undelive red bytes with restart-safe tombston es	24 h arti-fact · 7 d report · 30 d history

✓

Why this table is intentionally demanding

The table exercises authored widths, dense layout, status-badge wrapping, long links, numbers, dates, and multi-line recovery instructions. The exporter must preserve every value while making constrained layout decisions visible in its report.

Operational decision record

Accepted decisions

Decision	Rationale	Status
Keep PDF compilation local	Confluence content remains within the selected host and Atlassian session	ACCEPTED
Preserve unsupported macro bodies	A visible degradation is safer than silent content loss	ACCEPTED
Keep CLI exports in the fore-ground	The durable journal enables observation without pretend-ing a daemon exists	ACCEPTED

Final verification

⚠

Reference acceptance

The reference is accepted when the table of contents is navigable, the Mermaid diagram remains sharp at high zoom, code highlighting is visible, the control matrix retains every value, and the export report contains no unexplained degradation.

CONTENT PRESERVED

ARTIFACT VALIDATED

READY FOR REVIEW

END OF DOCUMENT

Reliable Documentation Delivery with atlcli

VERSION	v8
EXPORTED	July 25, 2026
PAGES	9